3.4.5 整体算法流程

算法 2: Hexa-Sweep 空区域提取、WKD-IBE 完备性密钥生成与 ZK 真实性证明

输入： TPC-H 输入文件，WKD-IBE 参数，Accumulator 参数，三维查询框 *query*

输出： 输出完备性密钥集合、几何完备性验证结果、真实性证明验证结果与 benchmark timing report

设置常量 $NumDims \leftarrow 3$, $BitLength \leftarrow 12$, $MaxKeyPatternDecryptChecks \leftarrow 64$

打印 `"Starting benchmark"`

初始化曲线参数

$(params, masterKey) \leftarrow$ WKDIBE.SETUP$(36, \textbf{true})$

$acc \leftarrow$ 加载 accumulator keys

构造三维查询框 *query*：

    $date \in [1994\text{-}01\text{-}01, 1994\text{-}12\text{-}31]$

    $discount \in [5, 7]$

    $quantity \in [0, 23]$

$dbData \leftarrow [\,]$, $dbFr \leftarrow [\,]$, $I \leftarrow [\,]$, $X \leftarrow [\,]$

$queryUniquePoints \leftarrow \varnothing$

▷ 阶段一：读取数据库并划分查询内外点

for all TPC-H 文件中的每一行 **do**

    $point \leftarrow$ 从当前行解析得到的三维点

    将 *point* 加入 *dbData*

    $pointBits \leftarrow$ FORMATPOINTTOBINARY$(point)$

    $fr \leftarrow$ SEEDTOFR$(pointBits)$

    将 *fr* 加入 *dbFr*

    if $point \in query$ **then**

        将 *fr* 加入 *I*

        将 *point.Coords* 加入 *queryUniquePoints*

    else

        将 *fr* 加入 *X*

    end if

end for

```
digest_DB ← CommitFakeG1(dbFr)
digest_X ← CommitFakeG1(X)

▷ 阶段二：将三维查询框转换为初始 wildcard pattern
initialPrefixes ← MapToIDs(query)
initialPatterns ← [ ]
for all prefix ∈ initialPrefixes do
    pattern ← FormatToWildcardPattern(prefix)
    将 pattern 加入 initialPatterns
end for

▷ 阶段三：Hexa-Sweep 空区域提取
permutations ← 三个维度的全部 6 种排列
combinedEmptyPatterns ← [ ]
for all permutation ∈ permutations do
    bitOrder ← generateBitOrderCustom(permutation, BitLength)
    patterns ← SubtractPointsOrdered(initialPatterns, dbData, bitOrder)
    将 patterns 中的所有 pattern 加入 combinedEmptyPatterns
end for
maximalEmptyPatterns ← selectMaximalPatterns(combinedEmptyPatterns)

▷ 阶段四：近似求解最小 completeness key cover
(coverCandidates, emptyPointCount) ←
    buildCoverCandidates(maximalEmptyPatterns, query, queryUniquePoints)
selectedCover ← greedySetCover(coverCandidates, emptyPointCount)
selectedCoverPatterns ← [ ]
for all candidate ∈ selectedCover do
    将 candidate.Pattern 加入 selectedCoverPatterns
end for

▷ 阶段五：真实 WKD-IBE completeness key 生成
rootKey ← KeyGen(params, masterKey, emptyAttrs)
```

$initialKeys \leftarrow$ DERIVEINITIALPATTERNKEYS$(params, rootKey, initialPatterns)$

$verificationEmptyKeys \leftarrow$
DERIVEEMPTYPATTERNKEYS$(params, initialKeys, selectedCoverPatterns)$

$totalKeyBytes \leftarrow 0$

for all $key \in verificationEmptyKeys$ **do**

$totalKeyBytes \leftarrow totalKeyBytes + |$MARSHAL$(key,$ **true**$)|$

end for

▷ 阶段六：客户端几何完备性验证

$totalQueryVolume \leftarrow 0$

for all $pattern \in initialPatterns$ **do**

$totalQueryVolume \leftarrow totalQueryVolume +$ CALCULATEVOLUME$(pattern)$

end for

$(coveredEmptyPoints, maxOverlap) \leftarrow$ VERIFYEMPTYREGIONPATTERNS$(query, selectedCoverPatterns, q$

$sampledKeyChecks \leftarrow$ VERIFYDERIVEDPATTERNKEYS$(params, verificationEmptyKeys)$

$realSpatialVolume \leftarrow |queryUniquePoints|$

if $coveredEmptyPoints + realSpatialVolume \neq totalQueryVolume$ **then**

报错：空区域覆盖与真实点数量不匹配，完备性验证失败

end if

▷ 阶段七：**ZK-Accumulator** 真实性证明

$I_{poly} \leftarrow$ POLYTREE(I)

$C_I \leftarrow$ 对 I_{poly} 生成 Pedersen commitment

$zkMemProof \leftarrow$ ZKMEMPROVER$(C_I, digest_X, transcript)$

$h_{mem} \leftarrow$ HASH$(zkMemProof)$

$zkDegProof \leftarrow$ ZKDEGCHECKPROVER(C_I, I_{poly}, h_{mem})

$ok_1 \leftarrow$ ZKMEMVERIFIER$(zkMemProof, digest_{DB}, C_I.Com, transcript)$

$ok_2 \leftarrow$ ZKDEGCHECKVERIFIER$(C_I.Com, zkDegProof, h_{mem})$

$ok_1 \leftarrow$ ZKMEMVERIFIER$(zkMemProof, digest_{DB}, C_I.Com, transcript)$

$h_{mem} \leftarrow$ HASH$(zkMemProof)$

$ok_2 \leftarrow$ ZKDEGCHECKVERIFIER$(C_I.Com, zkDegProof, h_{mem})$

if $\neg ok_1 \vee \neg ok_2$ **then**

    报错：真实性证明验证失败
end if

▷ 阶段八：输出 **benchmark** 结果
打印并记录：
    查询内真实点数量 $|I|$
    查询外点数量 $|X|$
    初始查询块数量 $|initialPatterns|$
    极大空区域块数量 $|maximalEmptyPatterns|$
    最终 completeness key 数量 $|verificationEmptyKeys|$
    completeness key 总字节数 $totalKeyBytes$
    空点覆盖数量 $coveredEmptyPoints$
    最大重叠次数 $maxOverlap$
    抽样密码学验证次数 $sampledKeyChecks$
    ZK membership 验证结果 ok_1
    ZK degree-check 验证结果 ok_2
输出最终 timing report
程序结束